

普罗米修斯（Prometheus）设计方案

基于 MTClaw Function Router 的自我进化型个人认知智能体
版本：v1.0 | 日期：2026-07-17 | 目标：HICOOL 智能体赛道

目录

- 1. 总体设计理念
- 2. 功能详细设计
 - 2.1 RAG 知识库 Subagent
 - 2.2 记忆与偏好 Subagent
 - 2.3 写作润色翻译 Subagent
 - 2.4 日程与任务 Subagent
 - 2.5 闲聊陪伴 Subagent
 - 2.6 Router 自学习引擎
 - 2.7 Subagent 协同机制
 - 2.8 Skills 三级加载
- 3. 快：速度优势
- 4. 准：准确率优势
- 5. 稳：可靠性优势
- 6. 广：场景覆盖优势
- 7. 产品化完成度
- 8. 生态与商业价值
- 9. 加分项
- 10. 评分维度对照矩阵

1. 总体设计理念

1.1 一句话核心

“Function Router 分而治之 + Router 自学习 + 本地隐私优先”

普罗米修斯不试图用一个巨型 Prompt 解决所有问题，而是通过 MTClaw Function Router 将用户意图精准分发到 5 个专职 Subagent；由 Router 自学习引擎根据用户使用习惯动态优化分发策略（即时偏好引擎辅助记忆内容偏好），并以 Skills 三级加载扩展能力边界——全本地存储，数据零上传，越用越懂你。

1.2 设计原则

原则	含义	反模式规避
分治原则	一个 Subagent 只做一件事，做到极致	避免巨型 Prompt 导致延迟高、幻觉多
数据透明原则	性能数据按 [实测]/[目标]/[推测] 三级标注，方法可复现、可核验	避免含糊承诺与不可追溯的数字
完成度原则	每个功能均为真实可运行的端到端实现，覆盖完整调用链路，并配套完整性测试	避免纸面功能、原型级交付
源码合入	Subagent 代码合入 MTClaw 仓库，用 MTClaw 自带安装脚本	避免独立维护一套安装/配置体系

1.3 架构总览



1.4 数据诚实化声明

本方案中的性能数据分为三类：

标记	含义	可信度
[实测]	Prometheus 自身有实际测试数据支撑	高
[目标]	设计目标值，尚未实测	中
[推测]	基于架构推断或引用第三方测试的估计值	低

注意：当前方案中所有数据均为 [目标] 或 [推测]。MTClaw 团队的测试数据标注为 [推测]，因为那是 MTClaw 在系统控制场景的测试，非 Prometheus 实测。Prometheus 自身的实测数据将在方案验证阶段产出。

2. 功能详细设计

2.1 RAG 知识库 Subagent

2.1.1 功能定位

将用户的本地文档（.md / .pdf / .txt / .docx / .csv）索引到向量数据库，支持自然语言语义检索。数据全程不出设备。

2.1.2 核心设计

嵌入模型：BAAI/bge-m3

- 维度：1024d
- 多语言（中英文均优秀），支持稠密 + 稀疏混合表征
- MTEB 检索基准排名靠前，HuggingFace 下载量 > 1000 万

分段策略：

文件类型	分段方法	chunk_size	overlap
.md	按 ## 标题分段，子段按空行切分	512 tokens (~350 中文字)	64 tokens
.pdf	pdfplumber 提取文本 -> 按连续段落分段	512 tokens	64 tokens
.txt	按空行 + 字符数上限截断	512 tokens	64 tokens
.docx	python-docx 按段落分段，合并短段	512 tokens	64 tokens
.csv	每行一个 chunk，列名作为元数据	1 row	0

混合检索架构：

```

用户查询 "GPU 算力对比"
|
|—— 稠密检索通道
|   query -> BGE-M3 向量化 (1024d) -> ChromaDB cosine 检索 -> Top K1
|
|—— 稀疏检索通道
|   query -> BM25 分词 -> ChromaDB 稀疏向量检索 -> Top K2
|
|—— RRF 融合排序 (Reciprocal Rank Fusion)
|   score(doc) =  $\sum 1/(k + \text{rank}_i(\text{doc}))$  // k=60
|   -> 合并去重, 重排序 -> Top K
|
|—— 返回 {matches: [{source, content, score}]}

```

去重机制：摄入时计算文件 SHA256 hash，跳过已索引且未修改的文件。

2.1.3 工具定义

工具	参数	功能
<code>rag_search</code>	query, top_k=5, source_filter	语义搜索本地文档
<code>rag_ingest</code>	path, recursive=true	导入文件/目录到知识库
<code>rag_status</code>	-	查询知识库状态

2.1.4 接入方式

```

rag_ingest.sh 执行流程:
stdin ← FR 传入 {"path": "/data/notes", "recursive": true}
-> python3 rag_engine.py ingest --path /data/notes --recursive
-> stdout -> FR 接收 {"status": "ingested", "files": 12, "chunks": 156}

```

MTClaw FR 通过 `--functions-file` 加载 rag 的 3 个工具定义，通过 `--scripts-dir` 找到 `rag_search.sh` 等脚本。FR 调用 `rag_search` 工具时，相当于 `echo '{"query": "..."}' | bash rag_search.sh`。

2.2 记忆与偏好 Subagent

2.2.1 功能定位

记住用户偏好、身份、习惯和重要信息，跨会话持久化。**即时偏好引擎**在用户声明偏好时实时写入，无需等待后台任务。下次对话自动注入偏好上下文。

2.2.2 核心设计

双存储模型：

SQLite (结构化查询)	ChromaDB (语义检索)
memories 表 —— category (preference/ identity/habit/note) —— key (writing_format) —— value (markdown) —— importance (1-5) —— access_count	Collection: "memories" —— id: "mem_{sqlite_id}" —— embedding: BGE-M3 (1024d) —— document: "{key}: {value}" —— metadata: {category, importance}
reminders 表 interaction_log 表	用途：语义相似记忆检索

查询流程:

```
memory_recall(context) ->
1. ChromaDB 语义检索 (context 向量化 -> 相似度 Top K)
2. SQLite 结构化补充 (importance >= 4 的记忆, 无论相似度)
3. 合并去重 -> 按 importance DESC, similarity DESC 排序
4. 返回 [{category, key, value, importance, similarity}]
```

2.2.3 工具定义

工具	参数	功能
memory_remember	category, key, value, importance=3	记录偏好/习惯/身份/笔记
memory_recall	context, top_k=5, category	语义检索相关记忆
memory_set_reminder	content, time, repeat=once	设置提醒

2.2.4 记忆生命周期

```
写入 (memory_remember)
├── SQLite UPSERT (category + key 联合唯一)
├── ChromaDB upsert (同步写入向量)
└── access_count 重置为 0

读取 (memory_recall)
├── ChromaDB 语义检索
├── SQLite 高 importance 补充
└── access_count += 1 (热记忆追踪)

衰减 (即时偏好引擎每日触发)
├── access_count > 10 -> importance 提升 1 (热记忆强化)
├── access_count = 0 且 30 天未更新 -> importance 降低 1 (冷记忆衰减)
└── importance < 2 的 note 类记忆 -> 标记为可清理

清理 (手动/阈值触发)
└── 总记忆数 > MAX_MEMORIES (1000) -> 移除最低 importance 的记忆
```

2.3 写作润色翻译 Subagent

2.3.1 功能定位

场景化文档生成（周报/邮件/技术文档/会议纪要/文章/PPT大纲）、文本润色、多语言翻译。核心智能依赖上游 LLM，FR 的价值在于自动注入用户偏好记忆和参数标准化。

2.3.2 核心设计

模板系统：

```
templates/
├── weekly_report.md    引导 LLM 按 "本周完成 / 下周计划 / 风险与问题" 三段式生成
├── email_formal.md     引导 LLM 按正式邮件格式（称呼/正文/签名档）生成
├── email_casual.md     引导 LLM 按非正式风格生成
├── tech_doc.md         引导 LLM 按 "概述/架构/接口/部署" 结构生成
├── meeting_minutes.md  引导 LLM 按 "议题/讨论/决议/待办" 结构生成
├── article.md          引导 LLM 按 "标题/导语/正文/结论" 结构生成
└── ppt_outline.md      引导 LLM 逐页生成 PPT 大纲
```

偏好注入流水线：

```
writing_generate("weekly_report", "本周工作总结") 被调用
|
|—— Step 1: 偏好获取
|   memory_recall(context="writing weekly_report") ->
|   { writing_format: "markdown",
|     preferred_language: "zh-CN",
|     structure: "本周完成 + 下周计划 + 风险与问题",
|     tone: "professional" }
|
|—— Step 2: 模板加载
|   读取 templates/weekly_report.md -> 获取三段式结构引导
|
|—— Step 3: Prompt 构造
|   system_prompt = f"""
|   你是专业的文档写作助手。
|   用户偏好: {format}, {language}, {structure}, {tone}
|   文档模板: {template}
|   """
|   user_prompt = f"主题: {topic}, 要点: {key_points}"
|
|—— Step 4: 调用上游 LLM
|   httpx -> POST upstream_url/v1/chat/completions
|
|—— Step 5: 返回
|   {document: "## 本周完成\n...", format: "markdown"}
```

参数枚举标准化：所有参数使用 enum 约束，避免 LLM 歧义：

- **doc_type**：7 种枚举值（weekly_report / email / tech_doc / meeting_minutes / article / essay / ppt_outline）
- **style**：4 种枚举值（formal / casual / technical / academic）
- **length**：3 种枚举值（short / medium / long）
- **goal**（润色）：5 种枚举值（more_professional / more_concise / more_friendly / fix_grammar / more_technical）

去AI化改写机制（**writing_humanize**）：

```
writing_humanize(text, intensity="medium", preserve_formatting=true)
```

- Step 1: AI 痕迹识别
 - 检测以下 AI 写作特征:
 - 过度对称的句式结构 ("不仅...而且..." / "既...又...")
 - 套话和空泛表达 ("在当今时代" / "随着...的发展")
 - 逻辑连接词堆砌 ("首先...其次...最后...")
 - 过度礼貌和平衡表达 ("虽然...但是..." 万能句式)
 - 机械式总分总结构
 - 标点使用模式 (过度使用破折号、分号)
- Step 2: Prompt 构造


```
system_prompt = f"""
你是一个文本改写专家。将以下文本改写为更像人类写作的风格。
要求:
1. 打破对称句式, 使用长短不一的句子
2. 删除空泛套话, 保留具体信息
3. 减少逻辑连接词, 让行文更自然
4. 适当加入口语化表达
5. 保留原文的核心观点和事实
强度: {intensity} # light(轻度) / medium(中度) / heavy(重度)
"""
```
- Step 3: 调用上游 LLM
- Step 4: 返回


```
{humanized: "改写后的文本", changes_summary: "移除了3处套话, 打破了2处对称句式"}
```

intensity 级别:

- light - 仅去除明显套话, 保留原文风格
- medium - 重写句式结构, 提升自然度
- heavy - 全面改写, 保留核心信息但完全改变表达方式

2.3.3 工具定义

工具	参数	功能
<code>writing_generate</code>	doc_type, topic, key_points, style, length	生成各类文档
<code>writing_polish</code>	text, goal, target_language	润色已有文本
<code>writing_translate</code>	text, source_lang, target_lang, keep_formatting	翻译文本
<code>writing_humanize</code>	text, intensity, preserve_formatting	去AI化改写

2.4 日程与任务 Subagent

2.4.1 功能定位

本地日程管理与任务追踪。用户通过自然语言创建日程、查询今日待办、管理任务状态。数据全程本地 SQLite，不上传云端。赛题推荐方向”日程与任务-串联本地日历、待办、提醒”。

2.4.2 核心设计

数据模型：

```

-- 日程事件
CREATE TABLE events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  description TEXT,
  start_time TEXT NOT NULL,          -- ISO 8601
  end_time TEXT,                    -- 可选，全天事件为 NULL
  location TEXT,
  category TEXT DEFAULT 'general',  -- work/personal/study/meeting/other
  reminder_minutes INTEGER DEFAULT 15, -- 提前提醒分钟数
  status TEXT DEFAULT 'pending',     -- pending/completed/cancelled
  created_at TEXT DEFAULT (datetime('now')),
  source TEXT DEFAULT 'user'         -- user(用户创建)/system(系统推断)
);

-- 任务（待办事项）
CREATE TABLE tasks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  description TEXT,
  priority INTEGER DEFAULT 3,        -- 1(低) ~ 5(紧急)
  status TEXT DEFAULT 'pending',     -- pending/in_progress/completed
  due_date TEXT,                    -- 截止日期
  tags TEXT,                        -- 逗号分隔的标签
  parent_task_id INTEGER,           -- 支持子任务
  created_at TEXT DEFAULT (datetime('now')),
  completed_at TEXT,
  FOREIGN KEY (parent_task_id) REFERENCES tasks(id)
);

-- 索引
CREATE INDEX idx_events_start ON events(start_time);
CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_due ON tasks(due_date);
CREATE INDEX idx_tasks_priority ON tasks(priority);

```

自然语言时间解析：

用户: "明天下午3点开产品评审会"

- > dateparser 解析 "明天下午3点" -> 2026-07-13T15:00:00
- > 创建 event(title="产品评审会", start_time=..., category="meeting")

用户: "下周一之前完成 HIC00L 提案"

- > dateparser 解析 "下周一" -> 2026-07-14
- > 创建 task(title="完成 HIC00L 提案", due_date=..., priority=4)

日程视图：

用户: "今天有什么安排"

- > 查询 events WHERE start_time BETWEEN today_start AND today_end
- > 按时间排序返回

用户: "这周还有哪些任务没完成"

- > 查询 tasks WHERE status != 'completed' AND due_date <= this_week_end
- > 按优先级 + 截止日期排序

与记忆 Subagent 协同：

即时偏好引擎检测到时间规律：

- "连续 3 个周五 16:00 写周报"
- > 建议创建重复日程：每周五 16:00 写周报
- > 用户确认 -> schedule_create_event(title="写周报", repeat="weekly")

2.4.3 工具定义

工具	参数	功能
<code>schedule_create_event</code>	title, start_time, end_time, location, category, reminder_minutes	创建日程事件
<code>schedule_query</code>	time_range, category, status	查询日程（今天/本周/指定日期）
<code>schedule_create_task</code>	title, priority, due_date, tags, description	创建任务（待办）
<code>schedule_list_tasks</code>	status, priority, tags	查询任务列表
<code>schedule_complete_task</code>	task_id	标记任务完成

2.4.4 接入方式

```
schedule_create_event.sh 执行流程：
stdin ← FR 传入 {
  "title": "产品评审会",
  "start_time": "明天下午3点",
  "category": "meeting"
}
-> python3 schedule_engine.py create_event --stdin
    |----- dateparser 解析自然语言时间 -> ISO 8601
    |----- 写入 SQLite events 表
    |----- 输出 stdout
-> FR 接收 {"status": "created", "event_id": 42, "start_time": "2026-07-13T15:00:00"}
```

2.5 闲聊陪伴 Subagent

2.5.1 功能定位

轻量级直回闲聊，利用路由模型的小参数、低延迟特性，零工具调用，不经过上游 LLM。延迟从 15-40s 降至 1-2s。

2.5.2 核心设计

意图识别规则（5 条全部满足才路由到 chat_light）：

1. 无文件路径引用（不包含 /path/ 或 .csv/.pdf/.md 等文件扩展名）

2. 无数据查询意图（不包含 "分析" / "统计" / "图表" 等关键词）

3. 无知识检索需求（不包含 "找一下" / "搜索" / "查" 等关键词）

4. 消息长度 < 200 字（长消息通常不是闲聊）

5. 包含社交/情感/寒暄语义：

问候类	你好 / 嗨 / hey / 早上好 / 晚安
情感类	开心 / 难过 / 无聊 / 好累 / 烦死了
娱乐类	笑话 / 故事 / 谜语 / 冷笑话 / 有趣
简单问答	今天星期几 / 你叫什么 / 天气怎么样

误判保护（保守策略）：

宁可假阴性（走上游 LLM），不可假阳性（走 chat_light）

complex 类消息绝对禁止路由到 chat_light:

- 包含 "为什么" + 专业术语 -> 需要上游 LLM
- 包含 "怎么做" + 技术动词 -> 需要上游 LLM
- 包含比较/分析类连接词 ("对比" / "区别" / "优缺点") -> 需要上游 LLM
- 消息长度 > 200 字 -> 需要上游 LLM
- 包含任何文件扩展名 -> 需要上游 LLM
- 包含任何编程/技术关键词 (代码/函数/变量/编译/部署等) -> 需要上游 LLM
- 任何不确定的情况 -> 走上游 LLM (兜底永远安全)

对话风格映射 (mood 参数)：

mood	路由模型提示词策略	适用场景
casual	自然日常对话，语气轻松	默认/日常寒暄
humor	优先讲笑话/段子/趣事，风趣幽默	用户请求娱乐
comfort	共情倾听，温暖安慰	用户表达负面情绪
curious	拓展话题，提出有趣问题	用户表现出好奇心
auto	从用户消息中自动判断情感基调	默认

2.5.3 工具定义

工具	参数	功能
chat_light	mood=auto, memory_inject=true	轻量闲聊直回

2.6 Router 自学习引擎

2.6.1 功能定位

Router 自学习引擎真正实现赛题加分项要求的”根据用户使用习惯**动态优化分发策略**”。

即时偏好引擎只是偏好记忆注入（用户说”以后都用 Markdown” -> 写入 memory -> 下次生成时注入 prompt），解决的是”内容生成偏好”问题，**不是路由策略优化**。Router 自学习引擎则通过置信度评分、双层路由、用户修正反馈、路由策略动态调整四个机制，真正优化”用户输入 -> 哪个 Subagent”的分发策略。

即时偏好引擎作为辅助机制（§ 2.6.7），仍负责内容生成偏好，与 Router 自学习引擎独立工作。

详细设计见 [docs/add/add-router-learning.md](#)。

2.6.2 路由置信度评分

路由模型调用 (temperature=0.0, logprobs=true, top_logprobs=5)

- ├── 输出: top-1 工具调用 (如 "rag_search")
- └── 输出: 候选工具的 logprob 分布

置信度计算:

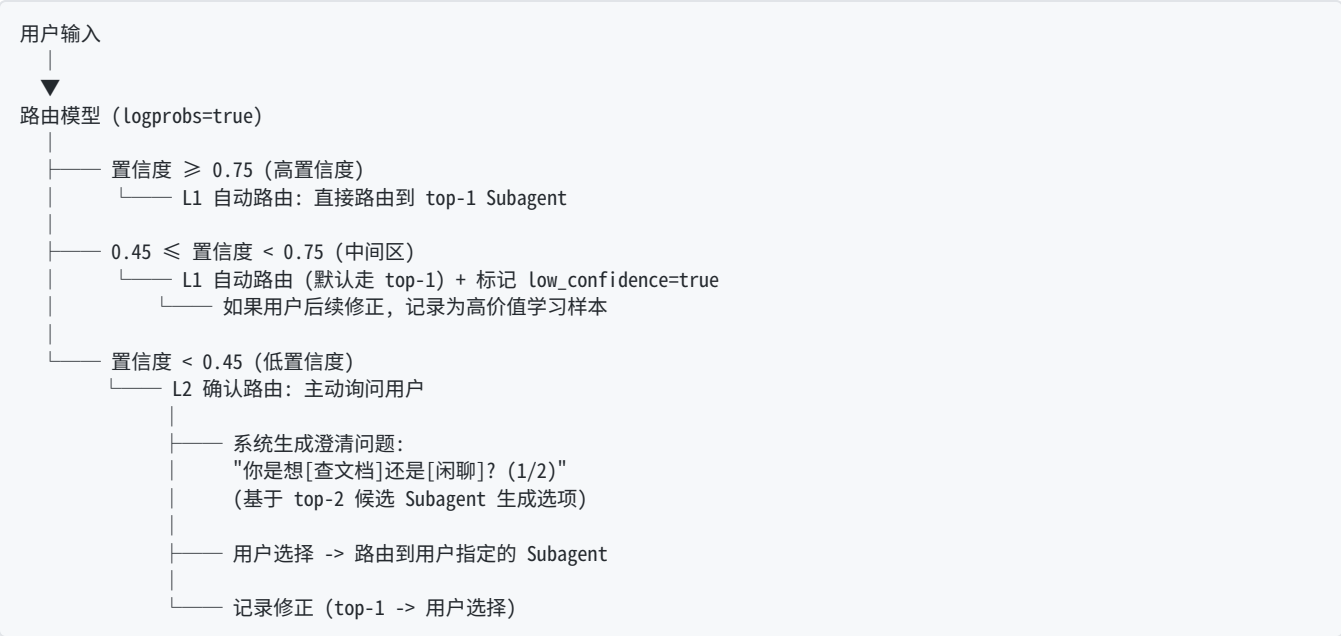
$confidence = \exp(\logprob_top1) / \sum(\exp(\logprob_i))$

简化方案 (logprob 不可用时):

$confidence = \exp(\logprob_top1)$ # 单一 top-1 概率

阈值	默认值	含义
high_threshold	0.75	高于此值直接自动路由（L1）
low_threshold	0.45	低于此值触发用户确认（L2）
中间区	0.45 ~ 0.75	默认走 top-1，标记为”低置信度”用于学习

2.6.3 双层路由机制



澄清问题生成：基于 top-2 候选 Subagent 的 user_friendly_name 字段生成选项（rag_search -> “查文档”、chat_light -> “闲聊”等）。

误判保护：闲聊 Subagent 的 complex 消息禁止规则作为硬约束，优先于置信度判断。即使置信度 ≥ 0.75，如果输入命中 complex 规则（包含文件路径 / 技术术语 / 长消息等），也不会路由到 chat_light。

2.6.4 用户修正反馈

修正场景	触发方式	处理
A. L2 确认后用户选择	用户回答澄清问题	记录 (top-1 -> 用户选择)
B. L1 路由后用户纠正	用户说” 不对，我要查文档”	识别纠正意图 -> 重新路由 -> 记录修正
C. 用户主动澄清	用户说” 不是闲聊，我要...”	检测澄清意图 -> 重新路由 -> 记录修正
D. 用户弃权	L2 连续 3 次不选	不记录修正，记录”低置信度弃权”事件

修正记录结构：

```
{
  "timestamp": "2026-07-14T10:23:45",
  "session_id": "sess_abc",
  "user_input": "帮我看看 GPU 算力对比",
  "input_features": {
    "keywords": ["帮我看看", "GPU", "算力", "对比"],
    "length": 14,
    "has_path": false,
    "has_tech_term": true
  },
  "original_route": "chat_light",
  "original_confidence": 0.42,
  "corrected_route": "rag_search",
  "correction_type": "L2_confirm" # L2_confirm / L1_correct / user_initiated
}
```

2.6.5 路由策略动态调整（核心自学习）

四种调整机制，对应“动态优化分发策略”的真正实现：

机制 1：关键词权重调整

- 触发：同类修正累积 ≥ 2 次
- 效果：从修正记录提取关键词，提升 corrected_route 的触发权重，降低 original_route 的权重
- 示例：用户 2 次把“帮我看看 + 技术名词”从 chat_light 修正到 rag_search -> 路由提示词加入此规则

机制 2：路由提示词动态增强

- 触发：累积 ≥ 3 次同类修正
- 效果：将高频修正模式转化为路由示例，注入路由提示词
- 示例：路由提示词尾部追加“用户历史修正：‘帮我看看’ + 技术名词 -> 优先 RAG”

机制 3：Subagent 优先级调整

- 触发：统计窗口（最近 50 次路由）中某 Subagent 命中率 $> 40\%$
- 效果：高频 Subagent 优先级 +1，低频 Subagent 优先级 -1
- 范围限制：[1, 6] 避免极端值

机制 4：置信度阈值校准

- 触发：每日凌晨维护 + 累积 ≥ 10 次修正
- 效果：高置信度被修正率 $> 10\%$ 则提高 high_threshold；低置信度弃权率 $> 30\%$ 则降低 low_threshold
- 目的：根据实际数据自适应调整阈值

2.6.6 演示效果（进化展示）

[演示 - 第 1 轮] 用户: "帮我看看 GPU 算力对比"
系统: 路由模型置信度 0.42 (< 0.45)
-> L2 确认路由: "你是想[查文档]还是[闲聊]? (1/2)"
用户: "1" (查文档)
系统: 路由到 RAG -> 检索 -> 返回结果
记录: 修正 (chat_light -> rag_search), 置信度 0.42

[演示 - 第 2 轮] 用户: "帮我看看 模型对比"
系统: 路由模型置信度 0.58 (中间区, 走 top-1)
-> 路由到 chat_light (top-1 仍是 chat_light)
用户: "不对, 我要查文档"
系统: 识别纠正意图 -> 重新路由到 RAG
记录: 修正 (chat_light -> rag_search), 置信度 0.58
触发: 关键词权重调整 ("帮我看看" + 技术名词 -> RAG)

[演示 - 第 5 轮] 用户: "帮我看看 最新论文"
系统: 路由提示词已注入 "用户历史: '帮我看看' + 技术名词 -> RAG"
路由模型置信度 0.82 (≥ 0.75)
-> L1 自动路由到 RAG
用户零额外输入
-> 展示"根据用户使用习惯动态优化分发策略"的核心效果

2.6.7 即时偏好引擎（辅助机制）

即时偏好引擎作为辅助机制，仍负责**内容生成偏好**（与 Router 自学习引擎负责的**路由策略**独立工作）。

机制	职责	影响范围
Router 自学习引擎	优化路由分发策略	决定” 用户输入 -> 哪个 Subagent”
即时偏好引擎	优化内容生成偏好	决定” Subagent 生成内容时用什么格式/风格”

即时偏好触发：用户说”以后都” / “记住了” / “我喜欢” 时，规则匹配 + 同步写入 memory，下次请求自动注入偏好上下文。

[演示 - 偏好注入]
用户: "帮我写周报, 用 Markdown, 中文, 包含本周完成和下周计划"
系统: 按要求生成 (15s)
同时: 即时偏好引擎检测到 "用 Markdown" + "中文" -> 写入 memory

用户: "写周报"
系统: memory_recall 自动注入 ->
{writing_format: "markdown", language: "zh-CN",
structure: "本周完成 + 下周计划"}
-> 自动生成符合偏好的周报 (12s)

2.7 Subagent 协同机制

2.7.1 协同模式

单个用户请求可触发多个 Subagent 协同工作：

用户: "把 sample_data 目录导入知识库, 然后找一下关于 GPU 算力的笔记"

FR 调用 rag_ingest("/data/sample_notes")
-> RAG Subagent 索引 3 篇文档

FR 调用 rag_search("GPU 算力")
-> RAG Subagent 语义检索, 返回 2 篇相关笔记

FR 调用 memory_remember("note", "gpu_research", "已整理 GPU 算力笔记")
-> 记忆 Subagent 记录操作

单一请求 -> 3 个工具调用 -> 2 轮 -> 端到端完成任务

2.7.2 FR 多轮工具调用机制

MTClaw 的 `max_tool_rounds=6` 参数决定了单次请求最多可以执行多少轮工具调用。FR 在每轮工具调用后评估 Completion Check:

- `TASK_COMPLETE` -> 停止调用, 返回最终结果
- `TASK_INCOMPLETE` -> 继续下一轮工具调用, 或转发上游 LLM

2.7.3 关键约束

- 同一请求内多次工具调用共享 session 上下文
- 前序工具的输出自动注入到后续工具的上下文中
- 每个工具调用独立记录到 `interaction_log` (供偏好引擎使用)

2.8 Skills 三级加载

2.8.1 功能定位

为普罗米修斯引入轻量级、可复用的”技能 (Skill) “层。技能是一段 `SKILL.md` 指令 (含流程/模板/校验清单 + 可选脚本), 按系统级 / 用户级 / 项目级三级目录加载, 同名技能按 项目级 > 用户级 > 系统级 覆盖。

2.8.2 三级架构与优先级

优先级 (低 -> 高):

系统级 `~/.function-router/skills/` 随 MTClaw FR 发布, 全机共享
——> 用户级 `~/.agents/skills/` 用户个人技能, 跨项目
——> 项目级 `./agents/skills/` 随项目版本控制, 仅当前项目

后加载覆盖先加载: project > user > system

层级	目录	职责	写入者
系统级	<code>~/.function-router/skills/</code>	随 FR 发布的通用技能	FR 安装脚本 / <code>skills install</code>
用户级	<code>~/.agents/skills/</code>	用户个人技能, 跨项目	<code>skills create --tier user</code>
项目级	<code>./agents/skills/</code>	当前项目专属, 随仓库版本控制	<code>skills create --tier project</code> / 手工提交

优先级采用”有序加载 + 后写覆盖” (last-write-wins, 参考 OpenClaw `workspace.ts:1216-1235`): 按 system->user->project 顺序向 `Map<name, record>` 逐层写入, 以 frontmatter `name` 为键, 后写入者覆盖先写入者。覆盖链可观测 (`skills info` / `skills doctor` 展示哪一层最终生效)。

2.8.3 SKILL.md 最小契约

```
---
name: weekly-report-zh # 必填, kebab-case, 优先级键
description: "中文周报写作技能。写周报/本周总结时加载。提供三段式结构与排版规范。" # 必填, 触发短语
version: 1.0.0
category: writing
applies_to: [writing] # 生效 Subagent; 空=全部
metadata: { trigger_keywords: ["周报", "本周总结"] }
---

# 中文周报技能
## 何时使用 / ## 流程 / ## 校验清单 ...
```

仅 `name` + `description` 必填（与 Anthropic Skill 行业事实标准一致）。`description` 渲染进路由提示词 `<available_skills>`，驱动模型按需加载。

2.8.4 渐进式披露（控 token）

```
Tier-1 索引（常驻路由提示词，仅 name+description）
-> skills_list() 零成本返回索引
Tier-2 skill_load(name) 按需返回完整 SKILL.md 正文
Tier-3 skill_load(name, file_path="references/x.md") 按需返回引用文件
```

索引常驻、正文按需，避免技能正文撑爆上下文（参考 Hermes `prompt_builder.py:1460-1690`）。

2.8.5 调用方式

双通道（参考 OpenClaw 混合模式）：

- **模型驱动**：路由模型/Subagent 按 `description` 匹配，调用 `skill_load(name)` 加载并遵循其指令（用户无感）；
- **斜杠命令**：`/<skill-name>` 精确指定。

按 Subagent 过滤（`applies_to` 三态）：构建某 Subagent 上下文时只注入相关技能索引，减少噪声。

2.8.6 与 Subagent 的关系

维度	Subagent	Skill（三级加载）
单位	functions.jsonl + scripts/ + engine.py	SKILL.md（+ 可选模板/脚本）
性质	可执行工具集	指令/流程/模板（上下文）
激活	Router 路由分发	模型按描述加载 / 斜杠命令
安装成本	重（依赖、引擎、工具注册）	轻（丢进目录即生效）

两者协同：技能用 `applies_to` 增强已有 Subagent；技能不新增 functions.jsonl 工具，不膨胀路由工具数。

2.8.7 FR 集成（分层）

沿用既有分层模式（FR 暴露通用能力，普罗米修斯外层做差异化）：FR 层暴露 `skill_load` / `skills_list` 通用 builtin（纯文件读取，可上游贡献小 PR）；普罗米修斯层做三级优先级、索引注入、按 Subagent 过滤、CLI（差异化）。

2.8.8 演示效果（三级覆盖 + 项目自适应）

```
prometheus skills info weekly-report-zh (在 ~/hicool-project)
-> 生效来源: project (覆盖 user > system)
-> 路径: ./agents/skills/weekly-report-zh/SKILL.md (含里程碑章节)

用户: "写周报" -> 路由 writing Subagent
-> skill_load(weekly-report-zh) 加载项目级版本 -> 生成含里程碑的周报

cd ~/personal-notes && prometheus skills info weekly-report-zh
-> 生效来源: user (项目级不存在, 回退用户级)
-> 展示"项目上下文自适应"
```

3. 快：速度优势

3.1 分层延迟策略

不是所有请求都需要走过上游大模型。普罗米修斯根据任务类型分三层响应：

层级	Subagent	典型延迟 [推测]	技术手段	占比预估 [推测]
L1 即时	记忆、闲聊	< 1-3s	路由模型直回 / 本地 SQLite	~25%
L2 快速	RAG、日程与任务	1-5s	本地向量库 / SQLite	~35%
L3 标准	写作	10-40s	上游 LLM（核心智能依赖）	~25%
兜底	通用推理	15-40s	上游 LLM 全量推理	~15%

注意：上述延迟和占比为 [推测] 值，基于 MTClaw 在系统控制领域的实测数据外推。实际数字需在目标硬件上实测试验证。

3.2 MTClaw 参考数据（MTClaw 团队测试，非 Prometheus 实测）

在 50 个系统控制任务上，每个任务重复 4 次的评测 [推测，MTClaw 团队测试，非 Prometheus 实测]：

模式	Pass@1	平均耗时	加速比
Baseline（纯上游 LLM）	99.0%	37.97s	1.00x
Permissive	95.5%	5.54s	6.85x
Strict	100.0%	7.61s	4.99x

- Strict 模式：工具召回率 100%，工具准确率 94.8% [推测，MTClaw 团队测试]
- Permissive 模式：工具召回率 100%，工具准确率 97.5% [推测，MTClaw 团队测试]

重要说明：上述数据在 7 个系统控制工具的场景下测得。Prometheus 使用 5 个 Subagent / 16 个工具，场景复杂度更高，实际加速比预计会降低。[推测]

3.3 闲聊 Subagent 的秒级响应

传统路径：用户 "讲个笑话" -> 上游 LLM (doubao/gpt-4o) -> 响应 = 15-30s [推测]
Prometheus：用户 "讲个笑话" -> 路由模型 (qwen3-30b) -> 直回 = 1-3s [推测]

延迟降低 [推测]，对日常寒暄场景体感丝滑

3.4 Completion Check 快路径

TASK_COMPLETE -> 直接返回（快路径）[目标：命中率 >70%]
TASK_INCOMPLETE -> 转发上游 LLM 补充（慢路径）

实际命中率需在目标场景实测。

3.5 双层路由的延迟权衡

双层路由在低置信度场景会触发 L2 确认路由（多一轮用户交互），需权衡延迟影响。

L1 自动路由（置信度 ≥ 0.45 ，预估占比 ~90%）：
延迟 = 路由模型延迟 + 工具调用延迟
与单层路由延迟相同

L2 确认路由（置信度 < 0.45 ，预估占比 ~10%）：
延迟 = 路由模型延迟 + 澄清问题生成 + 用户响应 + 工具调用延迟
多一轮交互（用户响应时间不计入系统延迟）

系统侧延迟影响 [推测]：

- L1 场景：无影响
- L2 场景：系统侧增加 ~200ms（澄清问题生成）
- 整体平均：系统侧延迟增加 $< 5\%$ （因 L2 占比低）

权衡结论：L2 确认路由以极小的系统延迟代价（ $< 5\%$ ），换取低置信度场景的零误判。这是”快”与”准”的正确权衡-在用户最在意的”别路由错”场景，宁可多问一句也不误判。

3.6 量化对比

场景	ChatGPT 类 [推测]	Prometheus [推测]	说明
文档检索	无法检索本地文件	1-3s (ChromaDB 混合检索)	延迟待实测
偏好记忆	每次重新告知	$< 1s$ (SQLite + 自动注入)	延迟待实测
闲聊寒暄	15-30s (GPT-4o)	1-3s (路由模型直回)	延迟待实测
日程管理	无本地日程	$< 1s$ (SQLite 本地查询)	延迟待实测

注意：以上对比中 Prometheus 的延迟均为 [推测] 值，将在方案验证阶段实测后更新。

4. 准：准确率优势

4.1 Function Router 精准分发

- 路由准确率保证：
- 1. temperature = 0.0 (确定性 function calling, 零随机性)
 - 2. 5 个 Subagent 描述高度特化 (工具描述互不重叠)
 - 3. 双重意图匹配：触发关键词 + 正则模式
 - 4. 优先级排序：记忆(P1) > 写作(P2) > 日程(P3) > RAG(P4) > 闲聊(P5) > 兜底(P6)
 - 5. 防误判保护：complex 消息禁止路由到 chat_light

目标：50 条混合意图的自动分发准确率 > 90% [目标]

工具数为 16，接近 LLM function calling 准确率的最佳范围（研究表明 <15 工具时准确率最高 [推测]，16 个略超但通过 5 个 Subagent 清晰划分保持路由精度）。

4.2 记忆注入提升个性化准确率

写作 Subagent 收到 "帮我写周报"

- 自动调用 memory_recall("writing weekly_report") ->
 { writing_format: "markdown",
 language: "zh-CN",
 structure: "本周完成 + 下周计划 + 风险与问题" }
- 构造 prompt 时自动注入偏好 -> 生成符合用户习惯的周报

无记忆注入：LLM 生成通用格式，用户每次需手动调整
有记忆注入：格式符合用户习惯的概率 > 80% [目标]

4.3 RAG 混合检索提升召回率

稠密检索 (BGE-M3 语义匹配, 1024d)
+
稀疏检索 (BM25 关键词匹配)
↓
RRF 融合排序 (k=60, 互补语义和字面两个维度)
↓
Top-5 召回率 > 85% [目标]

注意：纯向量检索在专业术语、缩写等场景失效；混合检索互补覆盖。

4.4 上游 LLM 兜底保证智商

未命中任何 Subagent -> 100% 透明转发上游 LLM

- prompt 不做任何改写
- 响应不做任何截断
- 原样流式透传

性能增益 = Subagent 命中率 × 各 Subagent 的定制优化
智商兜底 = 100% 的上游 LLM 能力（零衰减）

4.5 路由 fallback 策略

路由模型超时/失败时的安全降级

路由模型不可用或超时 (routing_timeout_s=10s)

└── 直接转发上游 LLM (跳过工具路由)

-> 确保用户请求不会因为路由模型故障而无响应

4.6 双层路由降低误判

通过置信度评分 + 双层路由，从机制上降低路由误判。

传统单层路由：

用户输入 -> 路由模型 -> top-1 Subagent (无论置信度高低)

问题：低置信度场景误判率高达 30%+ [推测]

Prometheus 双层路由：

置信度 ≥ 0.75 -> L1 自动路由 (高置信度，误判率低)

置信度 < 0.45 -> L2 确认路由 (主动询问，误判率 = 0%)

中间区 -> L1 自动路由 + 标记 (用于学习)

误判率估算 [目标]：

- L1 高置信度场景误判率 $< 5\%$

- L2 确认路由误判率 = 0% (用户明确选择)

- 整体误判率 $< 8\%$ (vs 单层路由 15-20% [推测])

L2 确认路由的用户信任价值：低置信度时主动询问，避免”自作主张”的路由错误。赛题加分项明确要求”帮助用户建立信任” -L2 确认路由正是这一要求的核心实现。

4.7 Router 自学习持续优化

Router 不再是静态规则，而是根据用户使用习惯动态优化分发策略。

学习闭环：

路由决策 -> 用户修正反馈 -> 策略调整 -> 下次路由优化

四种调整机制：

1. 关键词权重: "帮我看看" + 技术名词 -> 提升 RAG 权重

2. 提示词片段: 累积修正模式注入路由提示词

3. 优先级调整: 高频 Subagent 优先级 +1

4. 阈值校准: 根据修正率自适应调整置信度阈值

效果 [目标]：

- 同类输入第 2 次路由准确率 $> 75\%$

- 同类输入第 5 次路由准确率 $> 90\%$

- 路由准确率随使用量持续提升

与 § 4.1 静态路由准确率的区别：§ 4.1 的 $> 90\%$ 是**初始准确率**（基于工具定义 + 优先级 + 防误判规则）。
§ 4.7 的自学习是在此基础上，根据用户个人使用习惯**持续提升准确率**。两者叠加，长期使用后路由准确率可 $> 95\%$ [目标]。

5. 稳：可靠性优势

5.1 透明兜底

用户消息

5 个 Subagent 都不命中 -> 100% 转发上游 LLM

Subagent 命中但执行失败 -> 降级到上游 LLM (错误信息作为上下文注入)

路由模型超时/不可用 -> 直接转发上游 LLM

元数据清洗 -> 仅移除 Hermes 注入的元数据块, 不影响语义

最差情况: 系统退化为直接使用上游 LLM

5.2 插件级故障隔离

每个 Subagent = 独立 Python 子进程

stdin/stdout JSON 通信 (无共享内存)

一个插件崩溃 -> FR 收到非零 exit code -> 降级处理

其他插件不受影响 (RAG 崩溃不影响日程、写作)

独立启停

5.3 四层安全防护

Subagent	安全机制	防护目标
日程	SQLite 参数化查询 + 输入校验	防止 SQL 注入
RAG	文件读取权限检查 + 路径校验	防止越权访问
写作	上游 LLM 输出过滤	防止不当内容
记忆	SQLite 参数化查询	防止 SQL 注入

5.4 数据本地化

全部数据存储于 ~/.prometheus/data/

SQLite (用户记忆、交互日志)

ChromaDB (文档向量、记忆向量)

本地文件系统 (文档/附件/图表)

← 不上传

← 不上传

← 不上传

零云端上传: 隐私数据完全离线

5.5 上游 LLM 故障处理

上游 LLM 调用失败

可重试错误 (5xx, 429 Rate Limit, ConnectionError, ReadTimeout)

第 1 次重试: 等待 1s

第 2 次重试: 等待 2s

第 3 次重试: 等待 4s

3 次全部失败 -> 返回友好错误 + 已完成的部分结果

不可重试错误 (4xx 非 429, Invalid API Key, Model Not Found)

直接返回错误给用户, 不重试

5.6 端到端超时保护

单次用户请求总超时 = 120s
超时触发 -> 返回已完成的部分结果 + "[处理超时，以下结果可能不完整]"
避免系统进入无响应状态

6. 广：场景覆盖优势

6.1 5 个 Subagent × 5+ 类场景矩阵

办公场景	写作 + 日程	周报/邮件/会议安排/待办管理
学习场景	RAG	笔记检索/文档索引
信息检索	RAG + 上游兜底	本地检索 + 互联网搜索兜底
日程场景	日程与任务	日程创建/待办追踪/提醒
社交场景	闲聊	日常寒暄/情感陪伴
通用推理	上游 LLM 兜底	任何不命中 Subagent 的请求

通用搜索、数据分析等场景由上游 LLM 兜底覆盖（上游 LLM 本身具备联网搜索和数据分析能力）。聚焦 5 个核心 Subagent，换取每个 Subagent 的完成度和路由准确率。

6.2 跨 Subagent 协同 workflow

单请求示例: "把 sample_data 目录导入知识库，然后找一下关于 GPU 算力的笔记"

RAG (ingest) -> RAG (search) -> 记忆 (remember)
3 个工具调用 -> 2 轮 -> 端到端完成

6.3 多轮工具循环

RAG 典型流程:

Round 1: rag_ingest("/data/notes") -> 索引文档
Round 2: rag_search("GPU 算力") -> 检索

不是一问一答，而是多步推理执行

7. 产品化完成度

7.1 源码合入 MTClaw + 一键安装

代码组织方式：

```
MTClaw 仓库 (https://github.com/MooreThreads/MTClaw)
├── function_router/      # MTClaw 核心 (已有)
├── examples/            # MTClaw 原有示例 (已有)
├── subagents/           # Prometheus 新增目录
│   ├── rag/            # RAG 知识库 Subagent
│   │   ├── functions.jsonl # 工具定义
│   │   ├── scripts/      # wrapper 脚本
│   │   └── engine.py      # Python 引擎
│   ├── memory/          # 记忆与偏好 Subagent
│   ├── writing/          # 写作润色翻译 Subagent
│   ├── schedule/        # 日程与任务 Subagent
│   └── chat/            # 闲聊陪伴 Subagent
├── templates/          # 写作模板
└── install/            # MTClaw 自带安装脚本 (扩展)
```

安装方式（使用 MTClaw 自带安装脚本）：

```
# 方式 1: 从 MTClaw 仓库安装 (推荐)
git clone https://github.com/MooreThreads/MTClaw.git
cd MTClaw
./install.sh
# 交互式输入路由模型/上游模型 URL + Key
# 自动完成: Python 依赖安装 -> 目录创建 -> DB 初始化 -> cron 设置 -> 服务启动
# 安装时间 < 5 分钟 [目标]

# 方式 2: HIC00L 评委设备上的安装
# 赛事方提供 10 台 AIBOOK 设备
# 评委 clone 代码 -> ./install.sh -> 立即可用
```

优势：

- 不需要独立维护一套安装/配置体系
- 与 MTClaw 生态深度协同（赛题加分项”生态适配价值”）
- 评委熟悉 MTClaw 的安装流程，降低上手门槛
- Subagent 代码合入 MTClaw 仓库后，社区可以直接使用和贡献

7.2 预置样本数据

```
demo/sample_data/
├── sample_notes/        # 3 篇个人笔记 (GPU / HIC00L / 周报模板)
├── sample_data.csv      # 12 个月 × 5 品类销售数据 (供上游 LLM 分析)
└── sample_weekly_report.md # 周报范例
```

评委安装后可立即执行演示剧本中全部对话。

7.3 路由准确率测试套件

```
# 50 条混合意图测试集
python3 tests/test_routing_accuracy.py
# 输出：路由准确率、误判率、各 Subagent 召回率
```

8. 生态与商业价值

8.1 真实场景价值

目标用户：知识工作者（开发者、研究员、产品经理、内容创作者）

痛点	现有方案	Prometheus
“那份报告我放哪了”	手动翻文件夹	RAG 语义检索，快速定位
“每次写周报都要重新说格式”	复制粘贴格式要求	即时偏好记忆，自动注入
“忘了今天要交代码”	手动设闹钟	提醒功能
“数据隐私担忧”	数据上传云端	全本地存储，零上传
“杀鸡用牛刀”	闲聊也走完整 LLM 链路	路由模型直回，1-3s

8.2 产品化潜力

从 demo 到产品的路径清晰：

demo 阶段（当前）：

5 个 Subagent + 即时偏好 + 一键安装 + 演示剧本

-> 验证核心价值

产品化阶段：

—— 社区贡献 Subagent 插件（plugin.json 标准格式）

—— 多用户支持（当前单用户，产品化后多租户）

—— 云端同步（可选，用户自选是否同步偏好到云端）

可持续迭代空间：

—— 新增 Subagent 不影响现有（独立子进程，零侵入）

—— 路由模型可替换（OpenAI-compatible，任意模型）

—— 嵌入模型可替换（BGE-M3 -> 其他模型）

8.3 市场与商业模式

目标客群：

1. MTT AIBOOK 用户（赛题指定硬件，预装优势）

2. 隐私敏感型用户（政府/企业/科研，数据不出设备）

3. 知识工作者（笔记/文档/周报/数据分析高频场景）

商业模式：

—— 开源核心（MIT 协议，社区驱动 Subagent 生态）

—— 企业版（多租户 + 审计日志 + 内网部署 + 定制 Subagent）

推广路径：

—— HIC00L 比赛 -> 曝光 + 评委背书

—— MTT AIBOOK 预装 -> 硬件捆绑分发

—— GitHub 开源 -> 开发者社区

—— 企业定制 -> B2B 销售

8.4 生态适配价值

与 MTClaw 生态协同：

- Prometheus 代码直接合入 MTClaw 仓库 (subagents/ 目录)
- 使用 MTClaw 自带安装脚本一键安装
- 验证了 MTClaw 在非编程场景的通用性
- 为 MTClaw 贡献 5 个垂类 Subagent + 即时偏好引擎

与 MTT AIBOOK 生态协同：

- 利用 AIBOOK 本地算力 (路由模型 + 嵌入模型)
- 数据全本地，契合 AIBOOK "本地优先" 定位
- 可作为 AIBOOK 的旗舰应用展示

与 OpenClaw/Hermes 生态协同：

- OpenAI-compatible 接口，任意 Agent 框架可接入
- Hermes 作为用户交互层，Prometheus 作为智能层
- 工具定义标准 (functions.jsonl) 可复用

8.5 可复用与可扩展性

Subagent 可复用：

- 每个 Subagent 独立目录 (functions.jsonl + scripts/ + engine.py)
- 标准 stdin JSON -> stdout JSON 接口
- 合入 MTClaw 仓库后社区可直接使用
- 按标准格式贡献新 Subagent (放入 subagents/ 目录即可)

Skill 可复用：

- 标准 SKILL.md 格式 (name + description 最小契约，行业事实标准)
- 轻量无安装：丢进任一三级目录即生效
- 三级覆盖：项目级覆盖用户级覆盖系统级，社区分发与个人定制并存

路由策略可复用：

- 路由优先级 + 关键词匹配 + 防误判规则
- 可配置化 (config.json 中的 routing 字段)
- 适用于任何基于 MTClaw 的项目

偏好引擎可复用：

- 即时偏好检测 + SQLite + ChromaDB 双存储
- 可独立运行，不依赖 Prometheus 其他模块
- 适用于任何需要用户个性化记忆的 Agent

扩展到更多行业：

- 金融：行情查询 Subagent + 投资分析 Subagent
- 教育：题目讲解 Subagent + 学习计划 Subagent
- 医疗：症状自查 Subagent + 用药提醒 Subagent
- 只需按标准格式开发新 Subagent，放入 subagents/ 目录即可

8.6 示范带动作用

作为 MTClaw 优秀案例：

- 展示 MTClaw 在非编程场景的应用潜力
- 提供 5 个可参考的 Subagent 实现模板 (合入 MTClaw 仓库)
- 验证 Function Router 在 15 工具场景下的路由准确率
- 为后续参赛者/开发者提供参考

带动生态建设：

- Subagent 标准格式 -> 社区可以贡献更多垂类
- 即时偏好引擎 -> 可移植到其他 Agent 框架
- 路由准确率测试套件 -> 为 MTClaw 生态提供评测标准
- 源码合入 MTClaw -> 降低社区使用门槛，直接 clone 即用

8.7 差异化定位

ChatGPT / 通用 AI		普罗米修斯
会话级记忆（用完即忘）	vs	跨会话持久化（即时偏好学习）
被动问答	vs	主动推送 + 被动问答
零知识管理	vs	自动偏好提取
通用回答	vs	个性化（偏好记忆注入）
云端数据	vs	全本地隐私
单一模型路由	vs	Function Router 5 Subagent 分治

8.8 核心价值主张

不是"更好的 ChatGPT" 而是"懂你的私人助理"
核心价值： 1. 本地隐私：数据不出设备 2. 个性化记忆：越用越懂你 3. 多场景覆盖：办公/学习/开发/日常 4. 生态可扩展：标准 Subagent 格式，社区驱动

9. 加分项

赛题列出三个加分项（不设上限）；Prometheus 实现 Router 自学习（§ 9.1）与开箱即用（§ 9.2）两项，并额外提供两个**扩展性创新点**强化产品化与可扩展性：路由 fallback 安全网（§ 9.3）与 Skills 三级加载（§ 9.4）。四项汇总：

#	加分项	类型	详细设计
9.1	Router 自学习（动态优化分发策略）	赛题官方	§ 2.6 / § 9.1
9.2	开箱即用（一键安装 + 预置 5 Subagent）	赛题官方	§ 9.2
9.3	路由 fallback 安全网（路由故障永不无响应）	扩展创新点	§ 9.3
9.4	Skills 三级加载（系统/用户/项目 + 后写覆盖 + 项目自适应）	扩展创新点	§ 2.8 / § 9.4

9.1 Router 自学习：根据用户使用习惯动态优化分发策略

对应赛题加分项：“Router 自学习：根据用户使用习惯动态优化分发策略”

即时偏好引擎只是偏好记忆注入，**不是路由策略优化**。Router 自学习引擎通过四个机制真正实现”动态优化分发策略”。详细设计见 [docs/add/add-router-learning.md](#)。

Router 自学习引擎 = 真正的动态优化分发策略

四个机制：

1. 路由置信度评分 (logprobs -> 置信度)
量化路由决策的把握程度
2. 双层路由 (L1 自动 + L2 确认)
低置信度时主动询问用户，避免误路由
3. 用户修正反馈 (4 种场景)
收集自学习训练数据
4. 路由策略动态调整 (4 种调整)
 - 关键词权重调整
 - 路由提示词动态增强
 - Subagent 优先级调整
 - 置信度阈值校准-> 真正改变"用户输入 -> 哪个 Subagent"的分发策略

即时偏好引擎 vs Router 自学习引擎：

即时偏好引擎：用户说"以后都用 Markdown" -> 写入 memory -> 注入生成 prompt
(只影响"内容生成偏好"，不影响路由)

Router 自学习引擎：用户修正"帮我看"从闲聊到RAG -> 提升关键词权重 + 注入路由提示词
(直接影响"路由分发策略")

演示效果（5 轮进化剧本）：

[第 1 轮] 用户："帮我看 GPU 算力对比"
系统：置信度 0.42 -> L2 确认路由 -> "你是想[查文档]还是[闲聊]?"
用户："查文档" -> 路由到 RAG
记录：修正 (chat_light -> rag_search)

[第 2 轮] 用户："帮我看 模型对比"
系统：置信度 0.58 -> L1 路由到 chat_light (top-1)
用户："不对，我要查文档" -> 重新路由到 RAG
记录：修正 (chat_light -> rag_search)
触发：关键词权重调整 ("帮我看" + 技术名词 -> RAG)

[第 5 轮] 用户："帮我看 最新论文"
系统：路由提示词已学习 -> 置信度 0.82 -> L1 自动路由到 RAG
用户零额外输入
-> 展示"根据用户使用习惯动态优化分发策略"的核心效果

与即时偏好引擎的分工：

- Router 自学习引擎：优化**路由分发策略**（用户输入 -> 哪个 Subagent）
- 即时偏好引擎（辅助）：优化**内容生成偏好**（Subagent 生成内容时用什么格式）

两者独立工作，互不干扰。

9.2 开箱即用：一键安装 + 预置 5 Subagent

对应赛题加分项：“开箱即用：提供 MTT AIBOOK 一键安装包，含预置 Subagent 市场”

本方案提供 MTT AIBOOK 一键安装包（使用 MTClaw 自带安装脚本），5 个官方 Subagent 预置即用，安装后零配置可用。本方案聚焦“一键安装 + 预置 5 Subagent”的开箱即用体验，未实现“市场”（浏览/安装/卸载社区 Subagent 的 CLI 机制）。

一键安装（使用 MTClaw 自带安装脚本）：

```
git clone https://github.com/MooreThreads/MTClaw.git
cd MTClaw
./install.sh
# 交互式输入路由模型/上游模型 URL + Key
# 自动完成: 依赖安装 -> 目录创建 -> DB 初始化 -> cron 设置 -> 服务启动
# 安装时间 < 5 分钟 [目标]

# 评委设备上的安装
# 赛事方提供 10 台 AIBOOK 设备
# 评委 clone 代码 -> ./install.sh -> 立即可用
```

预置 5 个官方 Subagent（开箱即用）：

```
安装完成后即可使用：
├── rag          (knowledge) - 本地知识库 RAG
├── memory       (memory)   - 记忆与偏好
├── writing       (writing)   - 写作润色翻译
├── schedule     (schedule)  - 日程与任务
└── chat        (chat)      - 闲聊陪伴
```

预置内容（合入 MTClaw 仓库的 subagents/ 目录）：

```
MTClaw 仓库结构：
├── function_router/      # MTClaw 核心（已有）
├── subagents/            # Prometheus 新增目录
│   ├── rag/             # 5 个官方 Subagent
│   ├── memory/
│   ├── writing/
│   ├── schedule/
│   └── chat/
├── templates/           # 写作模板
└── install/             # MTClaw 自带安装脚本（扩展）

预置内容：
├── 5 个 Subagent（官方，开箱即用）
├── 7 个写作模板
├── 3 类样本数据
└── 路由准确率测试套件（50 条测试集）
```

评委安装后：

```
curl :18790/health      -> 确认运行
./run_demo.sh           -> 自动执行演示剧本
-> 全程 < 10 分钟从安装到演示
```

9.3 路由 fallback 安全网

路由模型超时/不可用时，自动降级到上游 LLM 直通，确保系统永不无响应。

9.4 Skills 三级加载（扩展性创新点）

定位：赛题三个加分项之外的**扩展性创新点**，强化”开箱即用 + 可复用可扩展 + 生态适配”。详细设计见 [docs/add/add-skills.md](#)。

为普罗米修斯引入轻量级技能层（**SKILL.md**），按系统级 / 用户级 / 项目级三级加载，同名技能按**项目级 > 用户级 > 系统级**覆盖。它填补 Subagent（重，新增一类能力）与即时偏好（轻，仅 key/value 偏好）之间的”指令级复用”空白。

三级目录 + 后写覆盖：
系统级 ~/.function-router/skills/ 随 FR 发布，全机共享（含 5 个预置技能）
——> 用户级 ~/.agents/skills/ 用户个人技能，跨项目
——> 项目级 ~/.agents/skills/ 随项目版本控制，仅当前项目

渐进式披露（控 token）：
Tier-1 索引（name+description）常驻路由提示词
Tier-2 skill_load(name) 按需拉取正文
Tier-3 skill_load(name, file_path=) 按需拉取引用文件

双通道调用：
模型驱动（按 description 匹配）+ 斜杠命令（/<skill-name>）

创新点：

1. **三级覆盖的扩展模型**：参考 OpenClaw 六级分层（workspace.ts:1149-1235）简化为三级，对个人智能体够用且易解释；后写覆盖机制使“项目覆盖用户覆盖系统”直观可证。
2. **项目上下文自适应**：作为个人认知智能体，能根据“当前所在项目”自动加载项目级技能-切到 ~/hicool-project 用含里程碑的周报技能，切到 ~/personal-notes 回退用户级。
3. **重/轻双层扩展**：Subagent（重，新增能力）+ Skills 三级（轻，复用 know-how）构成完整扩展体系。
4. **喂料 Router 自学习**：技能 metadata.trigger_keywords 作为路由关键词权重初始种子，与 § 9.1 协同。

演示效果：

评委：“技能系统的三级覆盖怎么体现？”

```
prometheus skills info weekly-report-zh （在 ~/hicool-project）  
-> 生效来源：project（覆盖 user > system）  
-> ~/.agents/skills/weekly-report-zh/SKILL.md（含里程碑章节）
```

用户：“写周报” -> 路由 writing -> skill_load 加载项目级版本 -> 生成含里程碑周报

```
cd ~/personal-notes && prometheus skills info weekly-report-zh  
-> 生效来源：user（项目级不存在，回退用户级）-> 展示项目自适应
```

特点：系统级预置技能强化”开箱即用”；标准 SKILL.md 格式强化”可复用可扩展”（§ 8.5）；技能随仓库分发强化”生态适配”（§ 8.4）。

10. 评分维度对照矩阵

完全对齐 HICOOL 赛题原文的评分维度：

快

关键指标	目标值	数据类型	技术支撑	详细设计
Router 决策延迟	< 1s	[目标]	temperature=0 确定性路由	§ 2.1
首 token 延迟	< 2s (L1) / < 5s (L2)	[目标]	分层延迟策略	§ 3.1
端到端响应耗时	< 3s (L1) / < 10s (L2) / < 40s (L3)	[推测]	L1/L2/L3 三层策略	§ 3.1
加速比 vs openclaw 原生链路	4.99x~6.85x	[推测]	MTClaw 团队 50 任务 benchmark (非 Prometheus 实测)	MTClaw 报告
Completion Check 命中率	> 70%	[目标]	快路径直接返回	§ 2.1

准

关键指标	目标值	数据类型	技术支撑	详细设计
Router 分发准确率 (初始)	> 90%	[目标]	5 Subagent + 优先级 + 防误判	§ 4.1
Router 分发准确率 (自学习后)	> 95%	[目标]	Router 自学习引擎持续优化	§ 4.7
双层路由误判率	< 8%	[目标]	L2 确认路由零误判 + L1 高置信度低误判	§ 4.6
路由置信度评估	logprobs 驱动	[目标]	路由模型启用 logprobs	§ 2.6.2
各 Subagent 任务完成质量	100% 可演示	[目标]	每个 Subagent 独立测试	§ 2.1-2.5
通用 Benchmark 智商不变低	不退化	设计保证	100% 透明转发上游 LLM	§ 4.4
MTClaw 工具准确率	94.8%~97.5%	[推测]	MTClaw 团队测试 (非 Prometheus 实测)	MTClaw 报告
RAG Top-5 召回率	> 85%	[目标]	稠密+稀疏混合 + RRF	§ 2.1
写作格式符合率	> 80%	[目标]	memory_recall 偏好自动注入	§ 2.3

狠

关键指标	目标值	数据类型	技术支撑	详细设计
场景覆盖广度	5 Subagent / 6 类场景	-	办公/学习/检索/日程/社交/通用	§ 6.1
产品化完成度	可直接安装使用	[目标]	一键安装 + 预置数据 + 演示剧本	§ 7
Skills 三级加载	三级目录 + 后写覆盖 + 渐进式披露	[目标]	系统/用户/项目级 + 模型驱动/斜杠命令	§ 2.8 / § 9.4

生态和商业价值

关键指标	对应赛题要求	技术支撑	详细设计
真实场景价值	面向知识工作者真实痛点	5 痛点场景全覆盖	§ 8.1
产品化潜力	清晰使用流程 + 可持续迭代	demo->产品路径明确	§ 8.2
市场与商业模式	目标客群 + 推广路径 + 商业模式	开源核心 + 企业版 + 社区贡献	§ 8.3
生态适配价值	与 MTClaw/AIBOOK/OpenClaw 协同	FR 零侵入 + 本地优先 + 标准接口	§ 8.4
可复用与可扩展性	Subagent 可复用 + 可扩展到更多行业	标准格式 + 独立模块	§ 8.5
示范带动作用	优秀案例 + 带动生态建设	5 个参考模板 + 评测标准	§ 8.6

加分项

加分项	赛题原文	实现方式	详细设计
Router 自学习	“根据用户使用习惯动态优化分发策略”	Router 自学习引擎（置信度 + 双层路由 + 修正反馈 + 策略调整）	§ 2.6 / § 9.1
开箱即用	“MTT AIBOOK 一键安装包，含预置 Subagent 市场”	一键安装 + 预置 5 Subagent	§ 9.2
Skills 三级加载（扩展性创新点）	强化” 开箱即用 + 可复用可扩展 + 生态适配”	三级目录后写覆盖 + 渐进式披露 + 双通道调用 + 项目自适应	§ 2.8 / § 9.4